SECURITY



OOB Impactless Update with Owner Authorization



Jayant Mangalampalli *Systems and Solutions, Intel Corporation*

Eli Kupermann *Security FW Architect, Intel Corporation*

Anahit Tarkhanyan *Xeon Product Architect, Intel Corporation*

Why impactless updates, why now

The cost today

- Reboots per update
- In-band load quiesces every core
- Multi-tenant scheduling stalls

Security impact

- Critical fixes wait for the next cycle
- Reserve pools shift downtime, not remove it
- Exposure window stays open

The payoff

- Continuous availability at runtime
- Security fixes without windows
- Interoperable across SoC fleets

Translating Challenge, R&R to Solution

Servicing Challenge

- Trust boundary shift:
  - Tenants, Platform owners, Si Vendors
  - Trust and Flexibility of handoff
- Key FW Asset
  - Microcode Update(MCU) - stages and activates via the BMC

Roles and Responsibilities

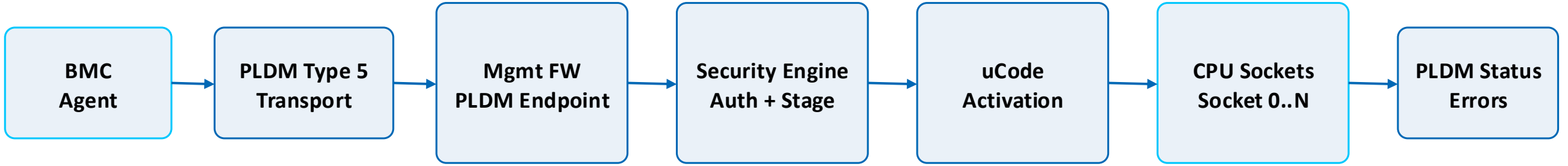
- Silicon vendors define the PLDM/OOB model and reference flows
- Operators own BMC, fleet policy, timing and rollback
- Owner have ability to authorize updates

Proposed solution — One signed servicing package carries payload, manifests and eligibility metadata.

I. BMC streams the package over PLDM and stages every socket
II. The owner authorizes the payload before activation
III. uCode activates under a platform-scoped rendezvous.

Let's Dive In

Trusted OOB Update path and uCode



Integration

- BMC owns the session
- Mgmt FW handles PLDM streaming
- Security Engine validates manifests

Correctness

- Per-socket staging
- One activation request per socket
- Platform-scoped rendezvous

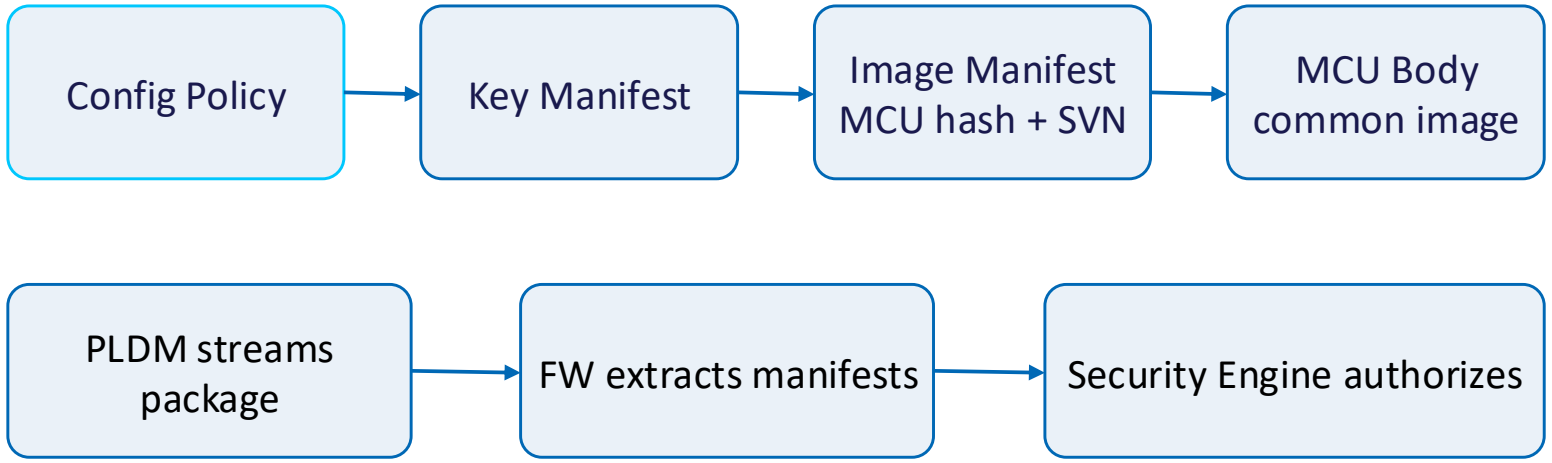
Visibility

- PLDM success/error reporting
- Eligibility in MCU metadata
- Retry for rejected activation

Owner Authorization: runtime authorization chain

What Owner Authorization adds

- Owner authorizes runtime microcode update
- Owner policy, “key authentication key” hashes and manifests define trust
- Security Engine checks MCU hash against authorized hashes
- Authorization occurs after staging, before activation



Combining OOB Impactless with Owner Authorization into one servicing package

PLDM firmware update image



Package creation

- Tools generate key and image manifests
- Tooling bundles manifests and payload into the PLDM image

Package processing

- Manageability FW fetches manifests from PLDM image
- FW streams data; cannot buffer full payload

Authorization decision

- Security Engine validates both manifests
- Manifest hash cached and compared at staging
- uCode not engaged if authorization fails

Design principle: Unauthorized package = Corrupted one

Call to Action

What we are asking OCP to converge on

- Collaborate on the OCP Impactless Firmware Update specification: contribute to review and shape future revisions
- A common eligibility metadata contract
- Common PLDM failure and status codes, with uniform version visibility across vendors

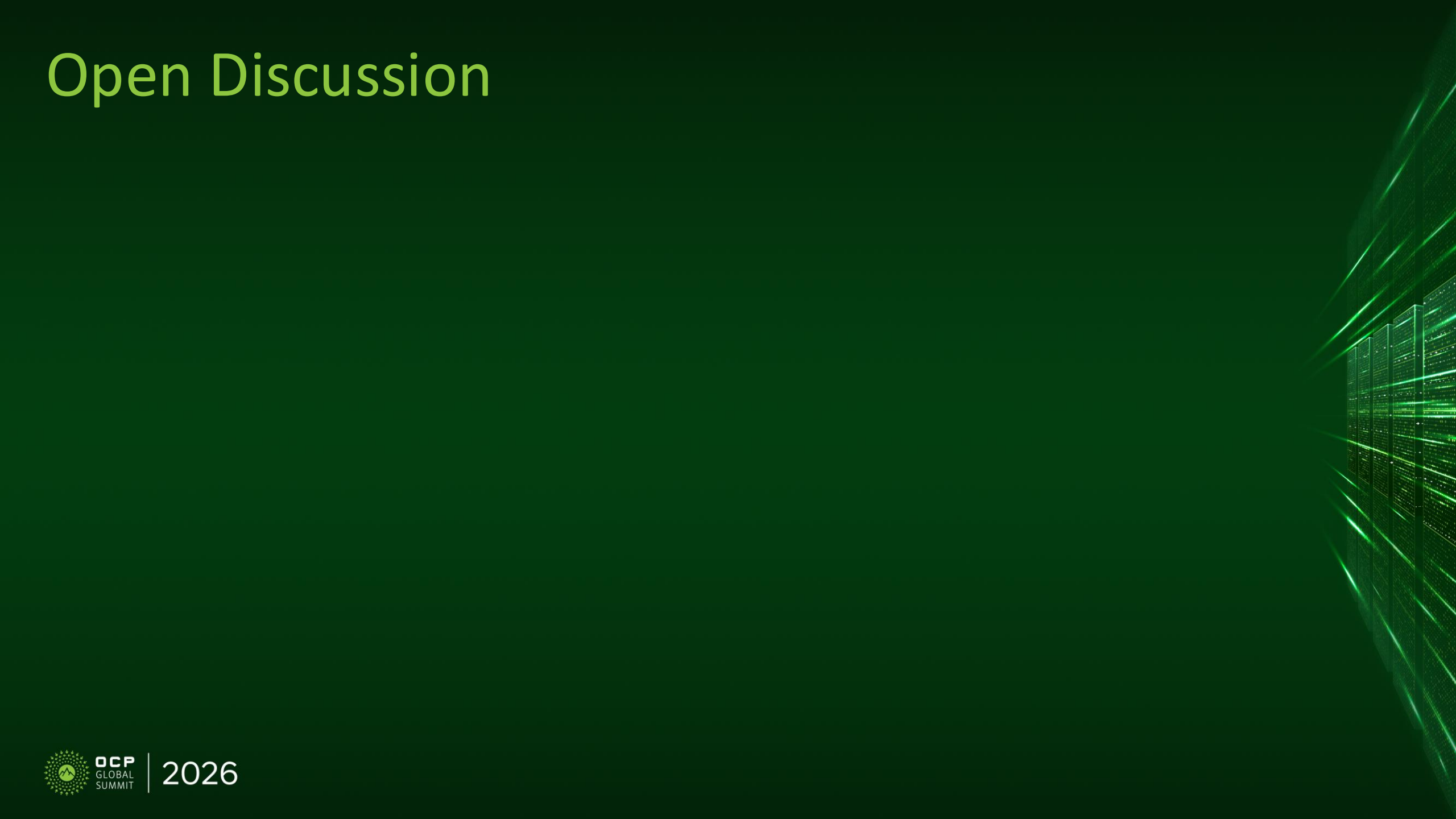
Boundaries that stay fixed

- Clear ownership of packaging tooling: who bundles manifests and payload into one PLDM image
- Operators keep timing, guardrails and rollback
- Silicon keeps correctness and hardware-safe activation

Thank You!



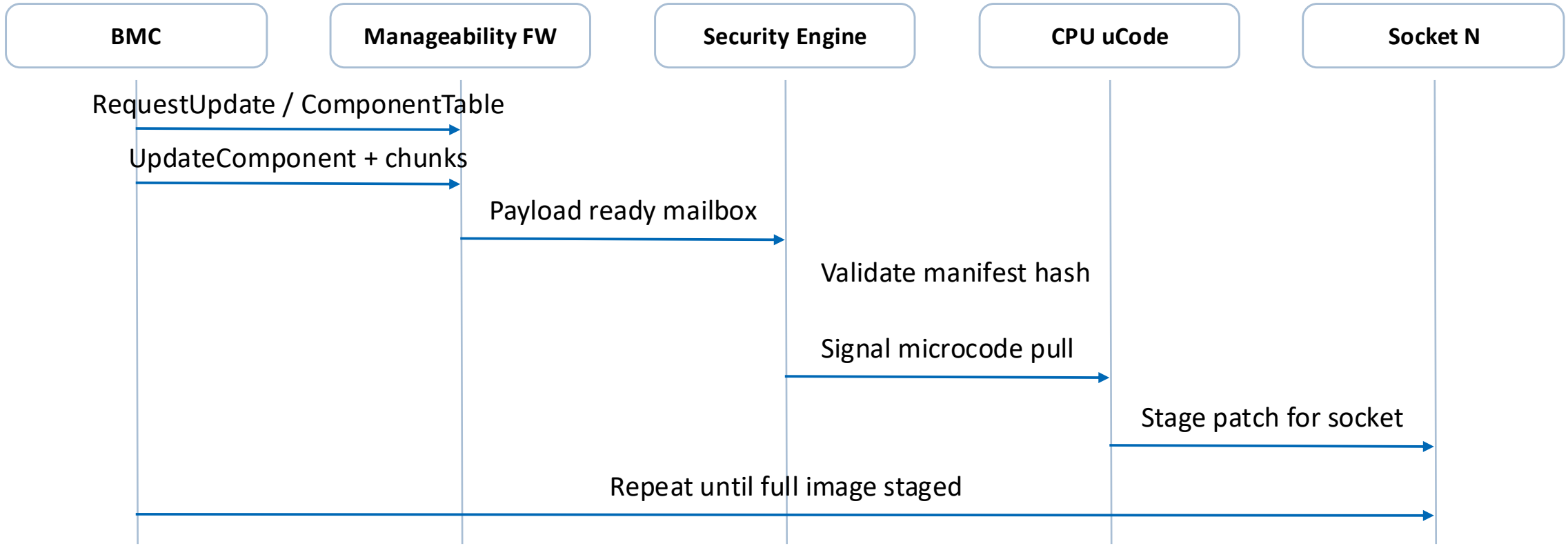
Open Discussion



Backup



Delivery, staging, and coordinated activation



Stage: every targeted socket fully staged before any activation; one common MCU patch stream.

Activate: per-socket PLDM Activate, platform-scoped rendezvous, one coordinated activation.

Eligibility policy: bounded-impact MCUs only

Header fields

- Minimum Runtime Update Revision: FW below it is too old
- Impact: 0 = impactless, 1 = impactful
- TCBR: encodes if TCB recovery is required
- Outcome: attempt, reject, or BIOS/reset path

Decision matrix

Min_Runtime	Impact	TCBR	Outcome
0 or N > current	x	x	Attempt; instability risk
N <= current	0	0	Attempt; no impact
N <= current	0	1	Invalid state
N <= current	1	0/1	Attempt; instability risk

OCP: standardize metadata so operators see if a package fits live OOB activation.

Supply chain and ownership of the servicing package

01

Silicon vendor

Signs unified MCU payload and SOC manifest

02

Platform OEM

Provisions policy flags and key hashes; owner-signs manifests

03

Authorizing vendor

Issues key and image manifests naming permitted MCU hashes

04

Packaging tooling

Bundles payload and manifests into one PLDM image

05

Operator and BMC

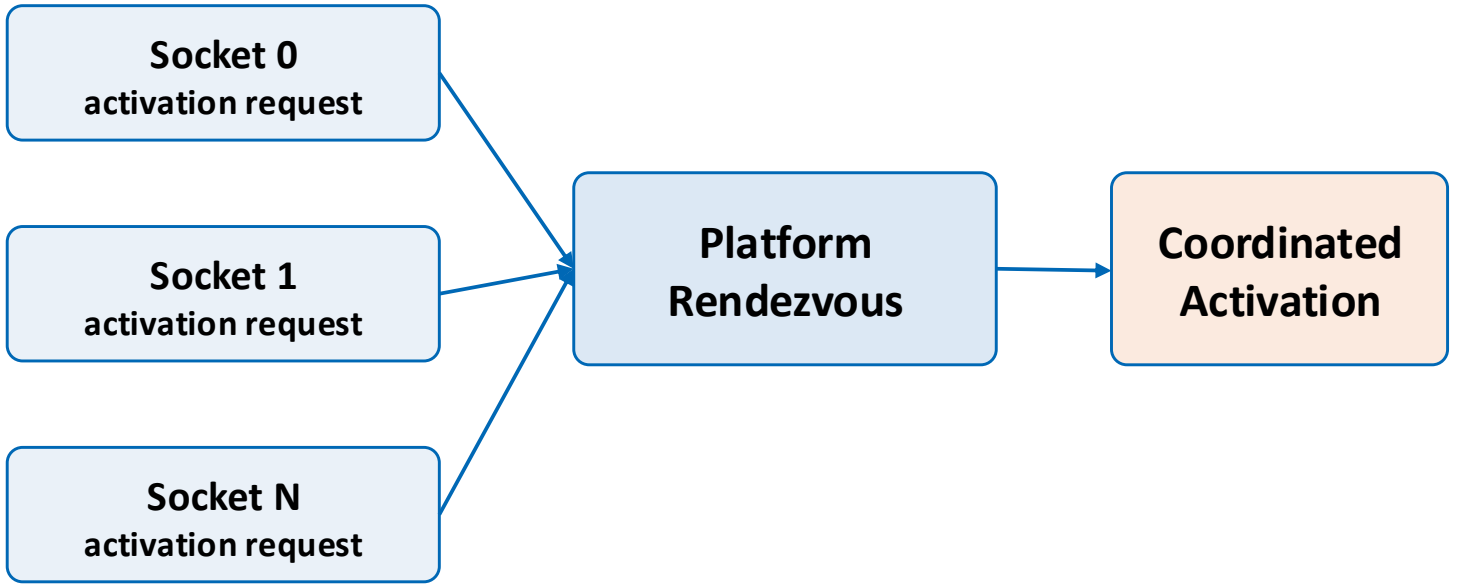
Delivers via PLDM, stages sockets, activates

Align on packaging tool ownership and a common eligibility metadata contract

Flow 2: Activation and rendezvous

Activation sequence

- BMC issues PLDM Activate
- Each CPU socket gets its own request
- Each socket enters platform-scoped rendezvous
- All sockets ready: one coordinated activation



Failure handling

Conflicting activity rejects the request; PLDM error flags report failure; OOB agent retries later.

Appendix: glossary of terms

Delivery and transport

- BMC: baseboard management controller; orchestrates the update
- OOB: out-of-band path, independent of the host OS
- PLDM: Platform Level Data Model, DMTF update transport
- MCU: microcode update, the CPU patch payload

Authorization and activation

- Manageability FW: on-die PLDM endpoint; streams the payload
- Security Engine FW: validates manifests, stages the payload
- uCode: microcode; hardware-safe activation
- Owner authorization: owner-controlled step before activation

Policy and metadata

- SVN: security version number; prevents downgrade
- TCBR: trusted computing base recovery; flags required recovery